懂吃 DongChi

检索增强的饮食决策系统

技术文档

参赛赛道：第九届传智杯 · AI 创新应用挑战赛（个人赛）

运行平台：HarmonyOS 6.0.2（API 22）｜ArkTS / ArkUI

版本：2.0.0

代码仓库：github.com/s1316339343/DongChi_V2.event_czb

文档版本：v2.0 · 2026-10-30

章节速览

01 摘要

02 一、问题定义与方案

03 二、技术架构

04 三、AI 能力设计（核心）

05 四、意图控制策略

06 五、如何约束 AI 生成的代码

07 六、可复现性论证

08 七、工程实现要点

09 八、实测数据

10 九、已知局限与后续规划

11 十、附录

摘要

"今天吃什么"每天要面对三次，难点不在"不知道有哪些菜"，而在约束条件下的取舍：时间、预算、冰箱库存、忌口，以及**自己做还是点外卖更划算。**

懂吃把这件小事做成了一套**决策系统**：三问渐进收敛条件 → AI 在确定性检索产出的合规候选内重排并解释 → 自己做与外卖的成本时间对比 → 冰箱与购物清单闭环。

技术上的核心主张只有一句：**硬约束由确定性代码保证，生成式模型只负责它真正擅长的语义理解与解释。**围绕这句话，本文重点回答三个问题：意图如何被控制（第四章）、AI 生成的代码如何被约束（第五章）、结论如何被复现（第六章）。

一、问题定义与方案

1.1 真正的痛点是取舍

约束	用户的实际处境	传统推荐应用的处理
时间	"只有 10 分钟"	照推 40 分钟的炖菜
预算	"20 元以内"	推荐 45 元的餐厅菜
库存	"冰箱里只剩鸡蛋和番茄"	不关心，让你去买
忌口	"不吃辣"	仍可能推麻婆豆腐
归属	自己做还是点外卖	只回答"吃什么"

1.2 方案：决策系统而非推荐工具

```
三问渐进式收敛条件 → AI 在合规候选内重排并解释 → 自己做 vs 外卖对比
                                                        ↓
                                              冰箱食材管理 + 购物清单闭环
```

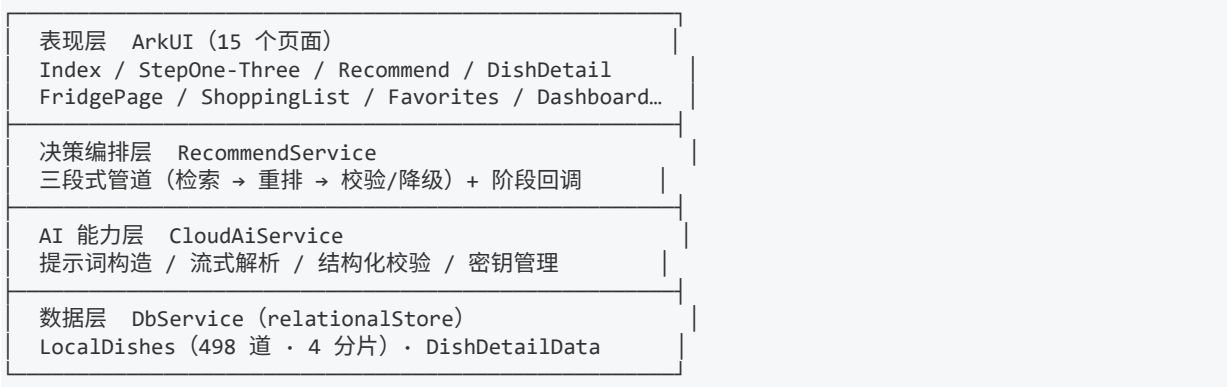
多数推荐应用停在第一步。懂吃对每道推荐菜同时给出双列信息（自己做 / 点外卖）的成本、耗时与食材齐备度，并直接给出判断，例如："你家里已有大部分食材，只需补买紫菜就能做！"

1.3 技术选型与理由

技术点	选型	理由
客户端框架	HarmonyOS ArkTS / ArkUI（API 22）	赛题平台；声明式 UI 适合卡片式信息密集布局
AI 接入	阿里云百炼（OpenAI 兼容接口）	免费额度覆盖开发与演示；兼容格式便于更换供应商
默认模型	`qwen-flash`	延迟低、成本低，适合"短输出 + 结构化"的重排任务
本地存储	relationalStore（SQLite）	决策记录/收藏/冰箱需要关系查询与去重，不适合 KV
配置存储	Preferences	轻量键值，用于 AI 密钥与开关
测试	hypium（12 个用例）	校验层是纯函数，可脱离网络做回归

二、技术架构

2.1 分层



分层的实际收益：`CloudAiService` 可以被整体替换（换供应商、改服务端代理）而不动车任何一个页面；`RecommendService` 里的本地引擎是完整可用的推荐器，不依赖网络。

2.2 关键接口（类型即契约）

结构	作用	边界意义
`CandidateDish`	送给模型的精简候选（名称/分类/时长/成本/所需食材/**家中已有 N/M**/本地分）	模型只能看到"已被证明合规"的集合
`RerankItem`	模型返回的 `{name, score, reason}`	只有这三个字段可被采纳
`RerankResult`	`{ok, items, failReason}`	失败必须给出可读原因，禁止静默
`RecommendResult`	`{dishes, isLocal, failReason}`	UI 必须知道"这是 AI 结果还是本地结果"

三、AI 能力设计（核心）

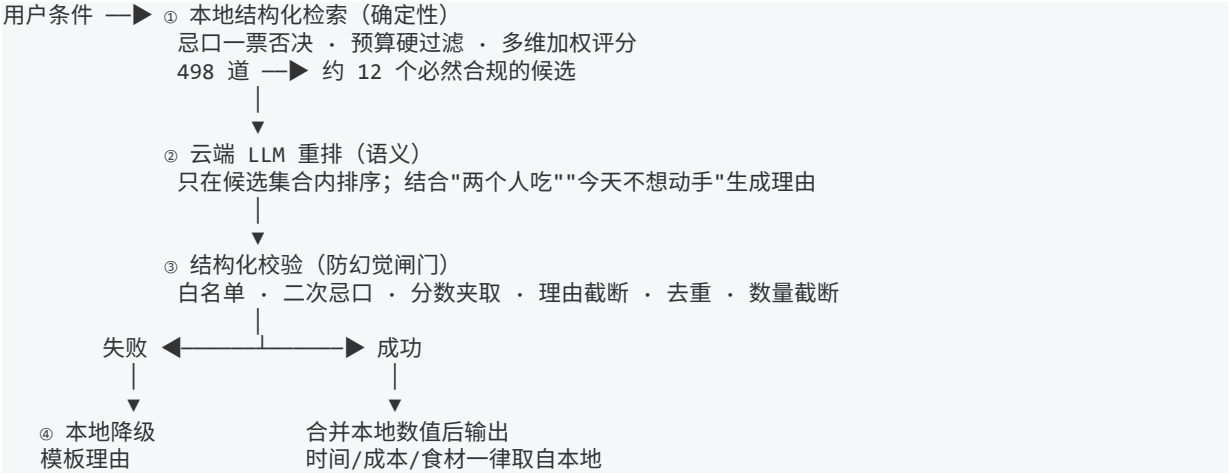
3.1 为什么不能"把条件直接丢给大模型"

早期实现是典型的**一次裸调用**：把用户条件拼成 prompt，让模型直接输出 3 道菜的 JSON。实测暴露三类问题：

1. **幻觉**：模型推荐菜品库里不存在的菜，下游无法展示详情、无法算成本； 2. **约束违反**：prompt 里写了"不吃辣"，仍可能推荐麻辣香锅——自然语言约束对生成模型不是硬保证； 3. **事实失真**：让模型自己编时间与价格，与本地数据不一致，用户看到的事实信息不可信。

根因：把「约束求解」和「语义理解」两类完全不同的任务交给了同一个组件。

3.2 三段式管道



3.3 数值与语义分离

模型只决定**排序**与**理由**；所有时间、成本、食材是否齐备一律由 `mergeRerankWithLocal` 从本地数据回填。

这条边界的价值在于：即使模型对价格判断完全错误，也不会污染用户看到的事实；反过来，模型擅长的"这句理由是否贴合处境"被完整保留。

3.4 降级设计（六条路径）

触发条件	行为	用户感知
未配置 API Key	纯本地推荐	降级提示条 + 说明原因

用户在设置中关闭 AI	纯本地推荐	同上
网络异常 / 超时（连接 8s、读 60s）	回退本地排序	弹窗说明
免费额度用尽（HTTP 403）	回退本地排序	错误翻译为"额度已用尽"
模型输出无法解析	自动重试一次（温度 0.3 → 0.1），仍失败则回退	无感（重试）或弹窗
校验后无有效结果	回退本地排序	弹窗说明

降级不是补丁，而是设计的一部分：本地引擎本身就是完整可用的推荐器，云端 AI 提供的是"更贴合的排序与解释"这一增量价值。

3.5 流式输出与决策过程可视化

早期的等待态是一个转圈的菊花灯。这在 AI 应用里是个具体的体验缺陷：用户不知道系统在做什么，也不知道要等多久。2.0 版本改为两件事：

1. **真流式**：`requestInStream` + `dataReceive` 增量解析 SSE，`TextDecoder` 开启 `stream` 模式（否则一个 UTF-8 汉字被切在两个分片里会乱码）；首个 token 即上屏。 2. **三阶段可视化**：管道向外汇报 `RETRIEVE → RERANK → VALIDATE / FALLBACK`，UI 用 ✅ / ⏳ / ○ / ⚠ 呈现每一步的真实状态；降级时**只把出问题的那一步标成** ⚠，不假装全部成功。

实测（真实 API 探测）：18 个分片、16 个增量、首片 375 ms、总耗时 608 ms，拼装结果 JSON 合法。首次尝试走流式，失败自动回退一次性请求并兼容"网关忽略 `stream` 参数"的情况。

四、意图控制策略

本章回答：如何把一个模糊的需求，变成 AI 能稳定产出、且可被自动验收的约束。

4.1 意图控制的三个层次

层次	手段	本项目中的落点
任务层	先拆解再执行	把"优化一下项目"拆成服务层/页面层/数据层三份只读审计
约束层	把模糊要求写成可判定的规则	忌口一票否决、预算硬过滤、候选上限 12、理由 ≤30 字
验收层	产物必须可自动检查	编译通过、单测通过、约束违反率 0、降级必须真的走到

4.2 系统提示词：四条硬规则

```
1. 只能从候选清单里选，禁止引入清单之外的任何菜名，禁止编造菜名。
2. 推荐理由要结合用户的具体处境（几人吃、冰箱里已有什么、时间是否紧张、预算是否敏感），
   不要说"营养美味""强烈推荐"这类空话。
3. 理由不超过 30 个字，直接给出判断依据，不要重复菜名。
4. 严格输出 JSON，不要输出任何解释文字、不要用 markdown 代码块包裹。


输出格式：{"items":[{"name":"菜名","score":92,"reason":"理由"}]}
```

设计意图：规则 1 把"生成"降级为"选择"，从根上消除幻觉空间；规则 2 强制模型使用上下文，否则退化成模板句；规则 3 约束长度，避免撑坏卡片布局；规则 4 降低解析成本。

4.3 用户提示词：把结构化条件自然语言化

结构化条件直接丢给模型效果差，因为模型对"timeRank=1"没有语感；本项目把它翻译成一段可读的处境描述，并**附带量化特征**：

```
- 口味方向：家常清淡
- 时间预算：30 分钟
- 花费预算：20 元以内
- 忌口限制：不吃辣
- 家里现有食材：鸡蛋、番茄、大米
- 用户补充说明：两个人吃，今天不太想动手
- 历史偏好：收藏过番茄炒蛋、蛋炒饭；最常选的口味方向是家常清淡


【候选菜品清单】（只能从这里选，共 8 道）
1. 番茄炒蛋｜家常清淡｜自己做约 10 分钟/8 元｜外卖约 25 分钟/18 元｜需要：番茄、鸡蛋（家中已有 2/2 样）
2. 清蒸鲈鱼｜家常清淡｜自己做约 25 分钟/28 元｜外卖约 30 分钟/45 元｜需要：鲈鱼、姜、葱（家中已有 0/3 样）
```

三个细节决定了输出质量：

- **"家中已有 2/2 样"**：把库存状态变成模型可直接引用的证据，理由是"食材全有"而不是泛泛的"简单易做"；
- **双列成本**（自己做 vs 外卖）：让模型有依据回答"哪种更划算"；
- **补充说明**：结构化条件无法表达"两个人吃""今天不想动手"，而这恰恰是模型能发挥价值的地方。

4.4 输出契约与重试策略

- **契约**：`{"items":[{"name","score","reason"}]}`，`score` 为 0–100 整数；
- **解析**：括号配对扫描抽取第一个完整 JSON 对象（跳过字符串内花括号、剥离 markdown 围栏），而非贪婪正则；
- **重试**：解析或校验失败后重试一次，温度由 0.3 降到 0.1（降低再次跑偏的概率）；
- **认证类错误不重试**：401/403 重试没有意义，直接进降级。

4.5 对 AI 编程助手的约束性提示

AI 结对开发的效率极高，但默认行为有三个隐患：**过度自信、无证据断言、改完不验证。**本项目固定了三条要求：

要求	具体表述	效果
要证据，不要断言	"每条结论必须给出文件 + 行号 + 原文"	审计报告从"建议优化"变成可核对的事实
不接受未验证交付	"每轮改动必须过一遍 hvigor 编译"	编译错误当场暴露，不堆积
不许伪造过程	"不得伪造 Git 时间戳；宁可如实说明历史缺失"	提交历史如实标注重建（见 RECONSTRUCTION.md）

4.6 意图 → 手段 → 可验证产出

意图	提示词/工程手段	可验证的产出
消除幻觉	只允许在候选内排序 + 白名单校验	幻觉压力测试零越界；12 个用例中的防幻觉用例
约束不泄漏	忌口双重校验（本地字段 + 返回后菜名扫描）	严格忌口场景零泄漏

事实不失真	数值与语义分离	模型给出的价格不进入 UI
失败不静默	六条降级路径 + 失败原因透传	无效密钥场景真的走到本地引擎
过程可信	会话记录导出 + 历史如实重建	`AI-PROMPTS/` 4 条链 + `RECONSTRUCTION.md`

4.7 人机分工实例（依据 AI-PROMPTS/ 会话记录整理）

决策	提出	定稿	理由
保留本地引擎作为降级，不砍掉	AI 方案	开发者采纳并写入验收标准	断网/额度用尽仍然可用
模型只做重排、不做生成	AI 根因推导	开发者采纳，落实为代码边界	幻觉零容忍
候选上限取 12	AI	开发者采纳	token 成本与语义空间的折中
删除局域网 Ollama 方案，改云端 API	开发者	开发者	装模型、改 IP、同 WiFi 门槛太高
密钥不入库	AI 提出	开发者执行，并扩展到签名口令	公开仓库合规
重建提交历史且如实标注	开发者	开发者	参赛要求与诚实之间的取舍

说明：本表由 AI 依据会话记录整理，用于说明"人在什么位置做判断"；具体措辞以开发者判断为准。

五、如何约束 AI 生成的代码

> 本章回答：代码是 AI 写的，凭什么说它可维护、没有严重漏洞。

5.1 AI 生成代码的三种典型退化

症状	表现	本项目的应对
越界生成	把"约束"写成注释而不是代码	硬约束必须是可执行的过滤逻辑，注释不算
局部最优	一个函数什么都干，越改越长	分层 + 单一职责（页面不写业务，服务不做 UI）
事实失真	用假数据、假价格、假状态把界面撑起来	数值一律来自本地数据；演示兜底数据不写库

5.2 边界约束：硬约束必须可执行

忌口判断不是写在 prompt 里的形容词，而是三步可执行逻辑：

```
① 本地字段过滤    dish.isSpicy / isSeafood / isVegetarian 直接剔除
② 云端二次扫描    返回菜名再扫一遍关键词（辣 / 香菜 / 虾蟹鱼贝…）
③ 校验闸门        任一项不合规即丢弃该项，全部丢弃则降级本地
```

预算同理：`budgetRank > budget` 的菜在进入候选之前就被剔除，模型看不到，也就无从违反。

5.3 四道闸门（`validateRerank`）

闸门	检查内容	不合格处理
白名单	菜名必须在候选集合内	丢弃（记录 warns）
二次忌口	菜名关键词命中忌口	丢弃
取值范围	`score` 夹取到 0–100，非法值给 60	修正
文本约束	理由压空白、截断 40 字（提示词约束 30 字，代码兜底 40 字）、空则填兜底句；菜名去重、结果截断为 3	修正/丢弃

5.4 可维护性约束

约束	具体做法	收益

单一职责	`CloudAiService` 只管调用与校验，`RecommendService` 只管编排与检索	换供应商不动页面
类型集中	`DishTypes.ets` 收敛共享类型与常量	改字段只改一处
常量抽取	食材分类从两份逐字重复抽成 `IngredientCategories.ets`	消除双写漂移
不可变共享数据	本地菜品库是共享对象，只读；只在最终选中时克隆 3 份	杜绝跨次推荐串数据
死代码治理	AI 迁移后遗留的空壳注释、未使用接口按缺陷处理并删除	后来者不会被误导

5.5 验收机制：证据 + 门禁 + 单测

机制	做法	结果
证据化	审计结论必须给文件+行号+原文	定位真实缺陷 80 余项并分级
编译门禁	每轮改动必须 `hvigor` 通过	编译错误不跨越轮次
单元测试	校验闸门是纯函数，脱网可回归	12 个用例覆盖全部边界
对抗测试	主动构造"模型最可能越界"的输入	幻觉压力测试零越界

5.6 一次真实的返工（保留不做美化）

> 抽取共享常量后忘记补类型导入，编译报错；测试文件使用无类型对象字面量，触发 ArkTS `arkts-no-untyped-obj-literals` 规则，需要按规则重构测试数据构造。

这类问题无法靠"更聪明的提示词"避免，只能靠编译门禁兜住——**这也是本项目坚持每轮编译的原因。**

六、可复现性论证

6.1 环境与依赖

类别	依赖	版本	说明
运行时	无第三方运行时依赖	—	`entry/oh-package.json5` 的 dependencies 为空
测试	`@ohos/hypium` / `@ohos/hamock`	1.0.25 / 1.0.0	devDependency
构建	DevEco Studio	≥ 6.1.1.280	提供 SDK / hvigor / JBR
构建	HarmonyOS SDK	6.0.2（API 22）	compileSdkVersion
构建	hvigor / Node / JDK	6.24.2 / 24.x / 21	随 DevEco 附带
可选	百炼 API Key	—	不配置也全功能可用

环境变量模板：`env.example.ps1` / `env.example.bat`（只含路径，不含密钥；`env.local.*` 已 gitignore）。

6.2 三条复现路径与实测

路径	步骤	实测
A 直接装包	从 Releases 下载 signed HAP → `hdc install`	约 1 分钟，不需要 IDE
B 源码编译	设环境变量 → `hvigorw assembleHap`	19–28 秒，`BUILD SUCCESSFUL`
C 跑测试	`hvigorw … -p module=entry@ohosTest` → Test 运行器	12 个用例，脱网可跑

6.3 过程可复现：从会话记录到提交历史

证据	位置	说明
原始对话	`AI-PROMPTS/`	4 条完整链，17 轮对话、537 次工具调用，逐轮原话与落盘文件
导出脚本	`tools/export-prompts.mjs`	多帧 zstd 解帧 + 打码，可按同一日志重跑复现
提交历史	仓库 24 个提交	1 个基线 + 11 个模块提交 + 12 个后续提交

重建说明	RECONSTRUCTION.md	说明为何重建、改了什么、局限在哪、如何回滚

关于历史重建的坦白：项目进入可用状态后才建仓，原始历史中第一次提交即包含全部功能，不符合"禁止一次性提交"的提交要求。因此把一笔大改动按模块拆成 11 个提交，时间取自会话记录的轮次起始时间，并在 RECONSTRUCTION.md 中明确标注"中间部分提交不保证可独立编译"。我们认为如实标注 + 保留全部原始记录，比伪造一串看似顺利的提交更接近"过程真实"。

6.4 自证命令清单

```
# 依赖与产物
git ls-files | wc -l                        # 79 个受控文件，无构建产物
grep -r "dependencies" oh-package.json5     # 运行时依赖为空


# 密钥不入库
git log -S "sk-" --oneline                  # 无输出 = 历史中无明文 Key
git check-ignore -v entry/src/main/resources/rawfile/ai-config.local.json


# 测试与构建
hvigorw assembleHap --mode module -p product=default -p buildMode=debug


# 过程证据
node tools/export-prompts.mjs --session <日志> --out /tmp/prompts
git log --oneline --reverse                 # 24 个提交，时间单调递增
```

七、工程实现要点

7.1 性能

优化项	优化前	优化后
候选生成	每次推荐全量克隆 498 道菜	只克隆最终选中的 3 道
权重计算	在 498 次循环内重复计算	循环外计算一次
结果挑选	两次 `filter` 构造中间数组 + 全量排序	单次线性扫描
决策记录页	每帧对每条记录 `JSON.parse`	加载时一次性解析并缓存
购物清单页	90 项图标表每次调用重建	提升为模块常量
仪表盘统计	`@State` 数组逐次 `push`+`sort`	本地构建后单次赋值

7.2 正确性

- **忌口双重校验**：本地字段过滤 + 云端返回后菜名二次扫描；
- **演示数据不污染统计**：AI 与本地均失败时的硬编码兜底菜不写入决策记录；
- **决策记录去重**：同一次决策内的"换一批"不重复写库；
- **偏好画像时效**：新增决策、删除记录、收藏变化均使画像缓存失效。

7.3 并发与资源

- TTS 监听器按 `requestId` 校验归属，丢弃过期回调，避免"图标已停但声音还在响"；
- TTS 初始化使用 in-flight Promise，防止并发初始化泄漏引擎实例；
- 流式请求在 `settle` 时统一 `off()` 事件并 `destroy()`，避免悬挂连接。

八、实测数据

8.1 端到端场景（真实调用，10-15 实测）

场景	验证目标	结果
基础场景（清淡/30 分钟/20 元/不吃辣/有鸡蛋番茄大米）	链路是否通	1.27 s 通过，869 tokens，理由正确引用"食材全有"
严格忌口（不吃辣 + 素食）	约束是否零泄漏	零泄漏，且主动避开候选里的麻婆豆腐
极紧预算（10 元以内）	预算硬过滤	推荐 5/7/9 元，均未越界
幻觉压力测试（候选全为白粥、水煮蛋等）	是否越界生成	未越界，只排给定三道
无效密钥	降级是否生效	401 → 不重试 → 回退本地引擎

8.2 流式与延迟（2.0 实测）

指标	数值
分片数 / 增量数	18 / 16
首个分片延迟	375 ms
整段耗时	608 ms
拼装 JSON 合法性	通过
编译耗时	19–28 s（`BUILD SUCCESSFUL`）
单元测试	12 用例全部通过

8.3 为什么这些数字可信

每一条都能被复现：场景 8.1 的输入条件在 `AI-PROMPTS/03-端到端验证与可追溯性.md` 里有逐轮原文； 8.2 的指标可用同源脚本或直接抓包重测；8.3 的编译与测试在本地可一键重跑。

九、已知局限与后续规划

局限	说明	规划
营养与步骤为模板化数据	498 道菜的步骤去重后约 300 条、营养仅 7 组区间，页面已标注"通用做法 / 仅供参考"	接入公开食物成分数据库逐菜补全
价格与时长是估算	外卖价格与配送时长无真实数据源	接入外卖平台开放接口或用户反馈校准
客户端持有密钥	反编译可提取	改为服务端代理（仅需替换 `baseUrl`）
使用已废弃 router API	编译存在弃用告警	迁移至 Navigation
长列表未虚拟化	收藏/记录/冰箱页使用 `Scroll`+`ForEach`	改为 `List`+`LazyForEach`
检索为规则打分	无向量检索	引入 embedding 做语义召回（当前规则已能保证零泄漏）

十、附录

A. 复现命令

```
Copy-Item env.example.ps1 env.local.ps1   # 改成本机 DevEco 路径
. .\env.local.ps1
& $env:HVIGOR_BAT assembleHap --mode module -p product=default -p buildMode=debug --no-daemon
```

B. 关键文件

文件	职责
entry/src/main/ets/service/CloudAiService.ets	提示词构造、流式解析、结构化校验、密钥管理
entry/src/main/ets/service/RecommendService.ets	三段式编排、本地检索、阶段回调、数值回填
entry/src/main/ets/pages/Recommend.ets	推荐页：三阶段可视化、流式面板、结果卡片
entry/src/ohosTest/ets/test/AiPipelineTest.test.ets	12 个校验闸门用例
tools/export-prompts.mjs	会话日志 → Prompt 快照导出脚本

C. 术语

术语	含义
候选（candidate）	本地检索产出、已通过全部硬约束的菜品集合
重排（rerank）	只改变候选顺序、不引入新菜名的模型调用
闸门（gate）	对模型输出做结构性与语义性校验的纯函数
降级（fallback）	任一环节失败时回退本地引擎，功能不中断
数值语义分离	模型只产出排序与文本，事实数据一律取自本地